Topics Discussed

1. Preemptive Scheduling
2. Nonpreemptive Scheduling
3. Scheduling Criteria
4. Scheduling Algorithms
   - i) FCFS
   - ii) SJF
   - iii) Priority
   - iv) RR
   - v) Multilevel Queue
   - vi) Multilevel Feedback Queue

Preemptive & Nonpreemptive Scheduling

CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state.
   - Ex: I/O request
2. Switches from running to ready state.
   - Ex: When an interrupt occurs
3. Switches from waiting to ready.
   - Ex: Completion of I/O
4. Terminates.

- Scheduling under 1 and 4 is *nonpreemptive*.
- All other scheduling is *preemptive*.

Scheduling Criteria

- ***CPU utilization***
  - Keep the CPU as busy as possible.
- ***Throughput***
  - No. of processes completed per time unit.
- ***Turnaround time***
  - Amount of time to execute a particular process.
  - Interval from time of submission to time of completion.
  - Sum of the periods waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.

Scheduling Criteria cont..

- ***Waiting time***
  - Amount of time a process has been waiting in the ready queue.
- ***Response time***
  - Amount of time it takes from when a request was submitted until the first response is produced, **not** output (for time-sharing environment)
- **Optimization Criteria**
  - Max CPU utilization
  - Max throughput
  - Min turnaround time
  - Min waiting time
  - Min response time

Scheduling Algorithms

- First-Come, First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- round-robin (RR) scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

First-Come, First-Served (FCFS) Scheduling

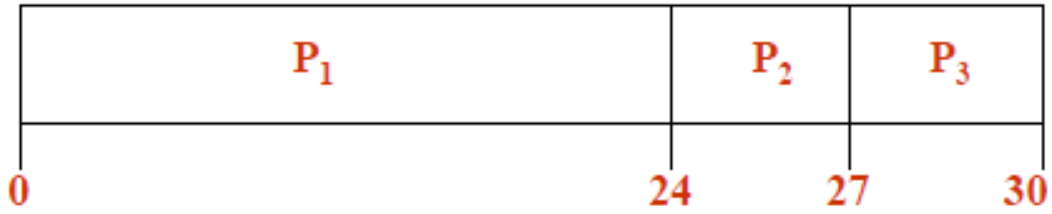
- Process that requests the CPU first is allocated the CPU first.
- FIFO queue.
- When a process enters the ready queue, its PCB is linked onto the tail of the queue.
- When the CPU is free, it is allocated to the process at the head of the queue.
- Nonpreemptive.

FCFS Scheduling Cont..

- **Ex**: Set of processes that arrive at time *0*.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
- **Gantt Chart:**



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$ ms

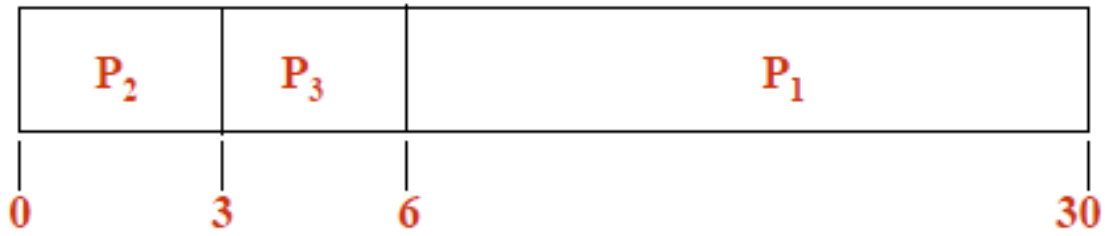
Formula to remember

- Response Time (RT) = (Time at which the process gets the CPU for the first time – Arrival Time)
- Waiting Time = (Turnaround time – Burst Time)
- Turnaround Time = (Waiting Time +Burst Time)
- Throughput (Efficiency of CPU) =Average Burst Time of all processes

FCFS Scheduling Cont..

Suppose that the processes arrive in the order P_2 , P_3 , P_1 .

- **Gantt chart:**



- Waiting time for $P_1 = 6$; $P_2 = 0$, $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case.
- ***Convoy effect*** **:** short processes behind one long process.

Shortest-Job-First (SJR) Scheduling

- Each process associates with the length of its next CPU burst.
- Lengths to schedule the process.
- When CPU is available, it is assigned to the process that has the smallest next CPU burst.
- If 2 processes - same CPU burst -use FCFS to break the tie.
- Optimal – gives minimum avg. waiting time for a given set of processes.
- Difficulty - knowing the length of the next CPU request.
- Used frequently in long-term scheduling.

Shortest-Job-First (SJF) cont…

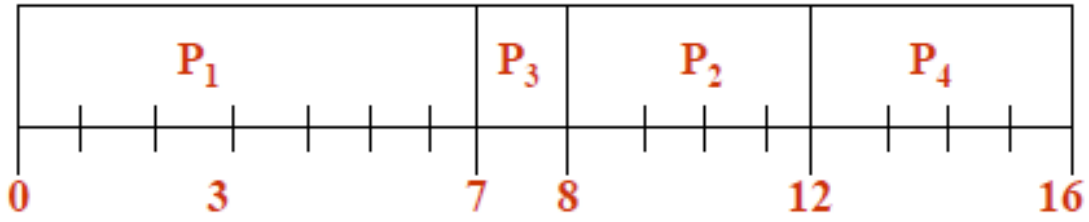
- May be either *preemptive* or *nonpreemptive.*

1. ***Non-Preemptive SJF***
   - Allow the currently running process to finish its CPU burst.
2. ***Preemptive SJF***
   - If a new process CPU burst less than remaining time of current executing process, preempt the currently executing process.
   - Also called ***Shortest-Remaining-Time-First (SRTF).***

Example of Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- **Gantt chart:**

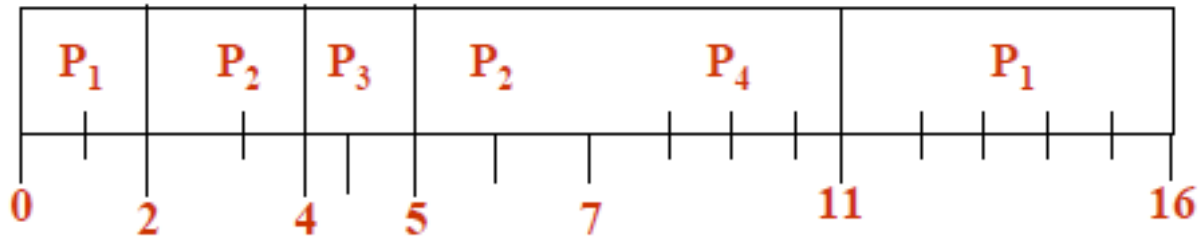


- Average waiting time = (0 + 6 + 3 + 7) /4 = 4

Example of Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- Gantt chart:



- Average waiting time = (9 + 1 + 0 +2)/4 = 3

Priority Scheduling

- A priority number is associated with each process.
- CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority).
- Equal-priority processes are scheduled in FCFS order.
- Either preemptive or nonpreemptive.
- Preemptive priority-scheduling
  - preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.
- Nonpreemptive priority-scheduling
  - Simply put the new process at the head of the ready queue.

Priority Scheduling cont..

- Priorities - Fixed range of numbers, such as 0 to 7, or 0 to 4,095.
- Priorities - defined either internally or externally.
- Internal priorities - use some measurable quantities.
- Ex:
  - Time limits
  - Memory requirements
  - Number of open files
  - Ratio of average I/O burst to average CPU burst .
- External priorities - set by criteria that are external to the OS, such as the
  - Importance of the process
  - Type and amount of funds being paid for computer use
  - Department sponsoring the work
  - Often political factors.

Priority Scheduling cont..

- **Example:** Set of processes P1, *P2,* ..., P5, arrived at time 0.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2



- Average waiting time is 8.2 milliseconds.

Priority Scheduling cont..

- **Problem** ≡ indefinite blocking or Starvation
  - Some low-priority processes waiting indefinitely for the CPU.
- **Solution** ≡ Aging
  - Technique
  - Increasing the priority of processes that wait in the system for a long time.

Round Robin (RR) Scheduling

- Designed for timesharing systems.
- Similar to FCFS, but preemption is added.
- ***Time quantum (or time slice)***
  - A small unit of time.
  - From 10 to 100 milliseconds.
- Ready queue is treated as a circular FIFO queue.
- Scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.
- Preemptive.

Round Robin (RR) cont..

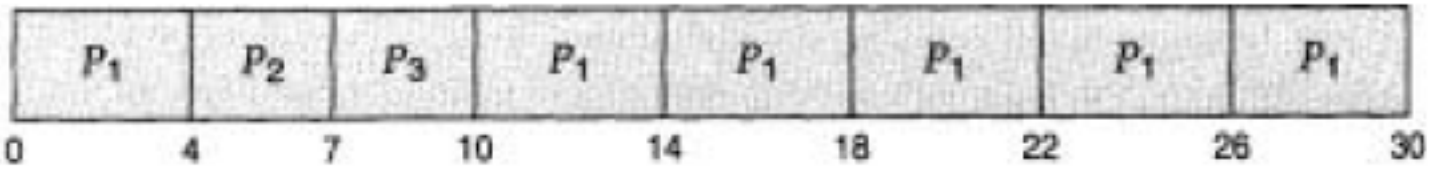
- Process - CPU burst <1 time quantum.
  - Process itself will release the CPU voluntarily.
  - Then scheduler will select the next process in the ready queue.
- Process - CPU burst >1 time quantum.
  - Timer will go off and will cause an interrupt to OS.
  - Context switch will be executed.
  - Process will be put at the tail of the ready queue.
  - Then scheduler will select the next process in the ready queue.

Round Robin (RR) cont…

- **Example:** Time quantum of 4 milliseconds

Process	Burst Time
P1	24
P2	**3**
P3	**3**

- **Gantt chart:**



- The average waiting time is ***17/3*** = 5.66 milliseconds

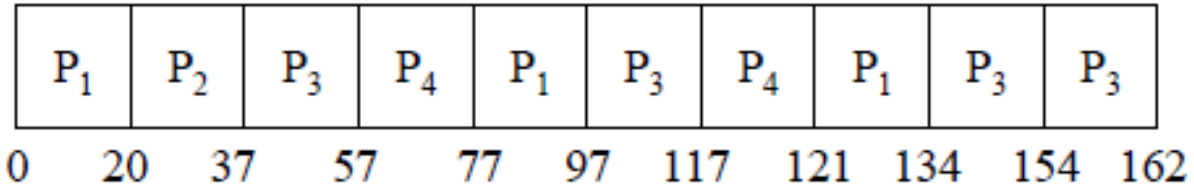
Round Robin (RR) cont..

- If there are
  - n processes in the ready queue
  - time quantum is q,
  - then each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
  - No process waits more than $(n\text{-}1)q$ time units.
- **Performance**
  - Depends on the size of the time quantum.
  - Time quantum is very large (infinite), RR policy = FCFS policy.
  - Time quantum is very small (say ***1*** microsecond), ***RR*** approach is called **processor sharing.**

Ex: RR with Time Quantum = 20

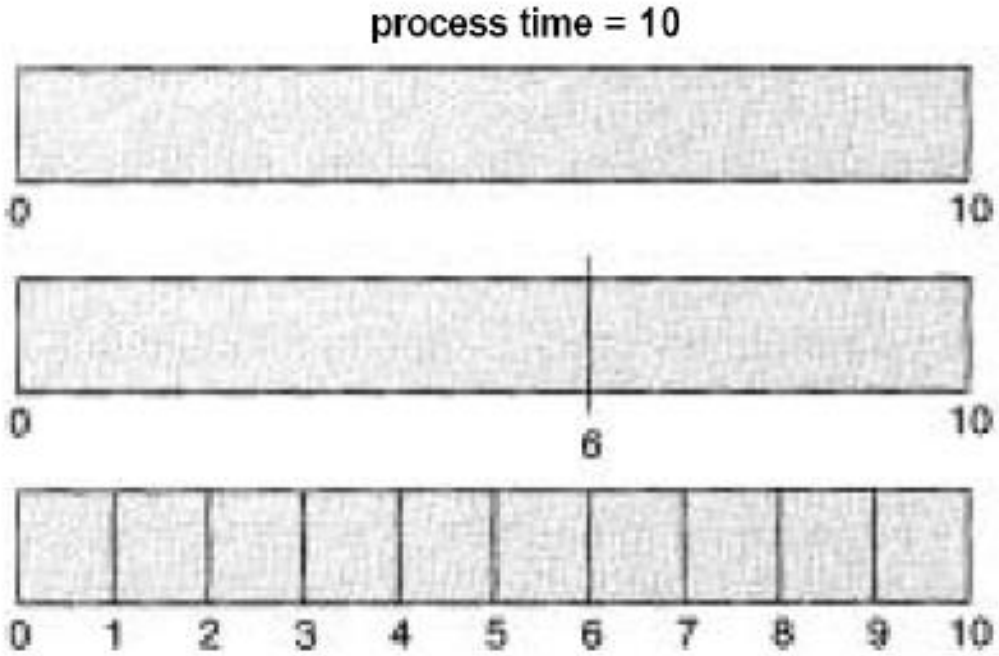
Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

- **Gantt chart:**



- Typically, higher average turnaround than SJF, but better *response*.

How a Smaller Time Quantum Increases Context Switches

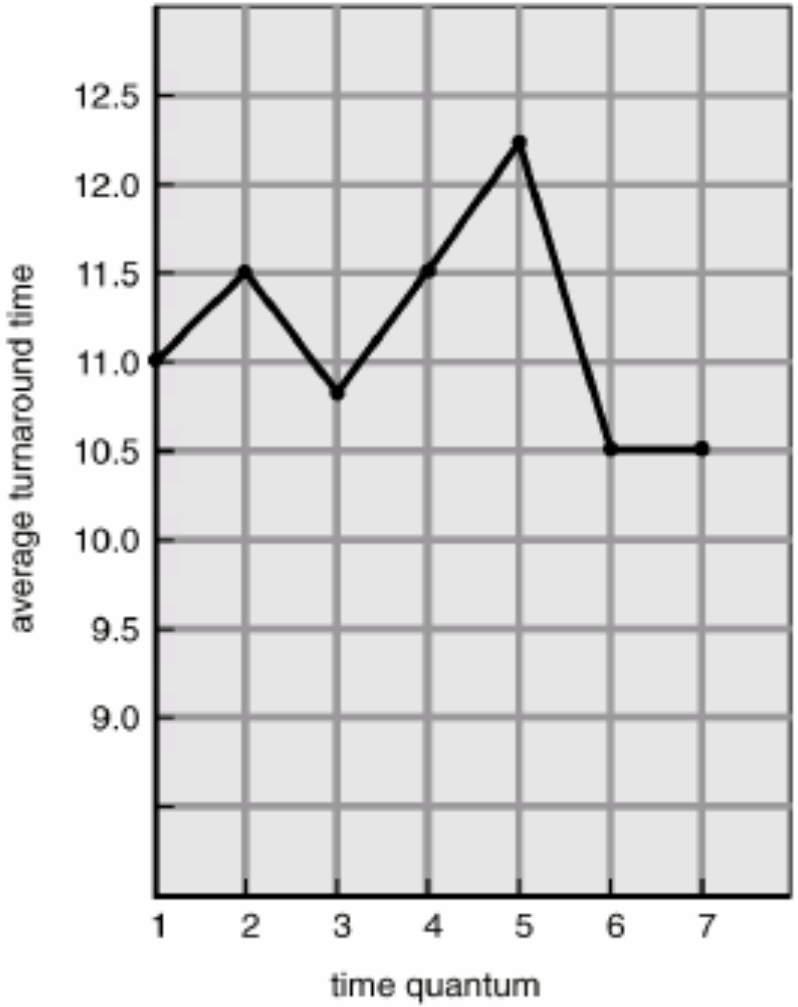


quantum	context switches
12	0
6	1
1	9

Turnaround Time Varies With The Time Quantum

- Turnaround time also depends on the size of the time quantum.
- Average turnaround time of a set of processes does not necessarily improve as the time-quantum size increases.

Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

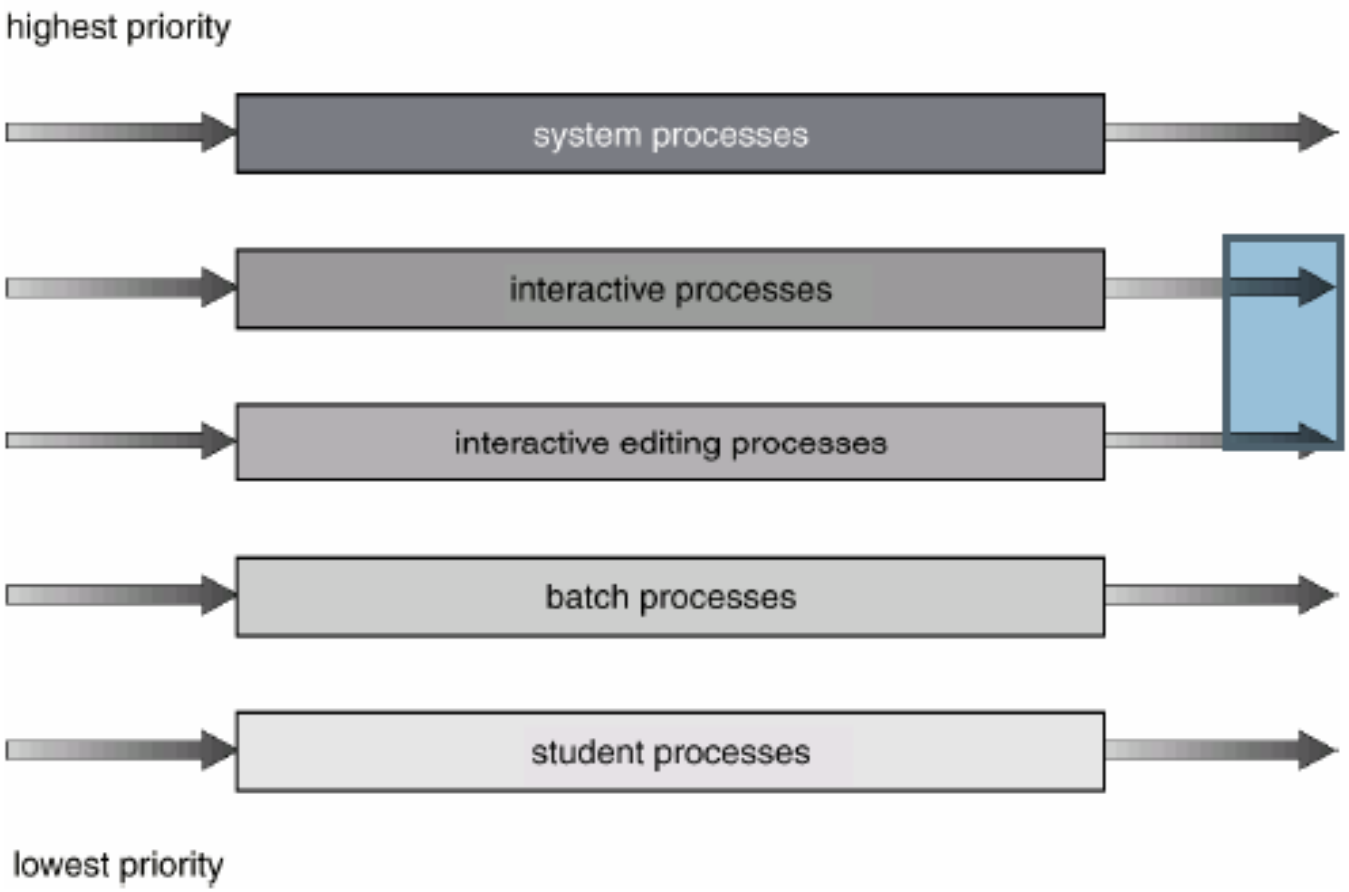
Multilevel Queue Scheduling

- Situation: Processes are classified into different groups.
- **Ex: Foreground** (or **interactive)** processes and **background** (or **batch)** processes.
- They have
  - Different response-time requirements
  - Different scheduling needs
  - Foreground processes may have priority over background processes.

Multilevel Queue Scheduling cont...

- Partition the ready queue into separate queues.
- Processes are permanently assigned to one queue, based on some property such as memory size, process priority, or process type.
- Processes do not move between queues.
- Each queue has own scheduling algorithm.
- Scheduling is implemented as fixed-priority preemptive scheduling.
- Advantage: low scheduling overhead.
- Disadvantage: Inflexible.

Multilevel Queue Scheduling cont…



Multilevel Queue Scheduling cont...

- **Ex:**
  - Separate queues used for foreground and background.
  - Foreground – RR
  - background – FCFS
  - Foreground queue have absolute priority over background queue.
- Possibility is to time slice between the queues.
- Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes;
- Foreground queue gives 80% for RR among its processes.
- Background queue gives 20% for FCFS.

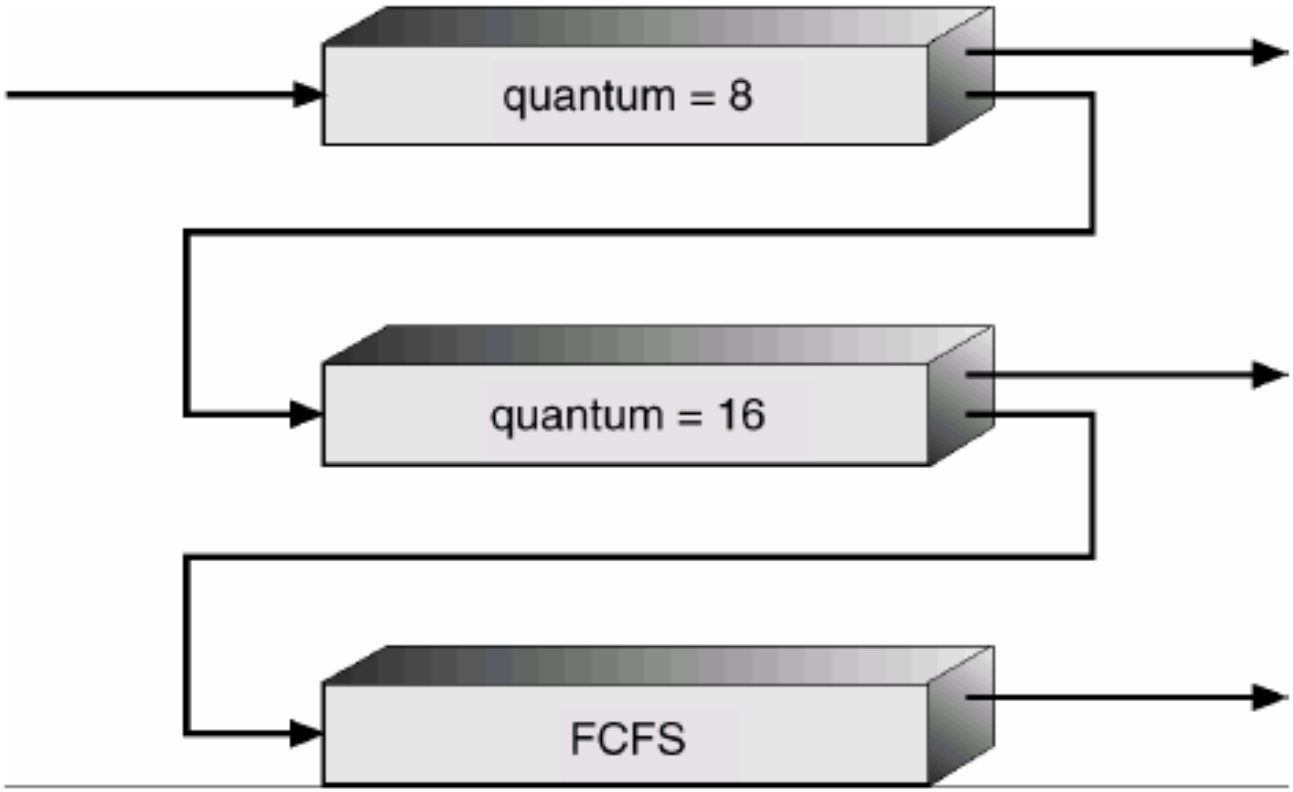
Multilevel Feedback Queue

- A process can move between the various queues.
- Separate processes with different CPU-burst.
- If a process uses too much CPU time, it will be moved to a lower-priority queue.
- If a process waits too long in a lower priority queue may be moved to a higher-priority queue- Aging prevents starvation.
- General CPU-scheduling algorithm.
- Most complex.

Multilevel Feedback Queue cont…

- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine
    - when to upgrade a process to a higher priority queue.
    - when to demote a process to a lower-priority queue.
    - which queue a process will enter when that process needs service.

Multilevel Feedback Queue cont…



Multilevel Feedback Queue cont…

- **Ex:** Consider
  - Multilevel feedback queue scheduler
  - Three queues, numbered from 0 to 2.
- Only when Q_0 = empty, execute Q_1.
- Only if queues Q_0 and Q_1 = empty. Execute Q_2 .
- Process arrives for Q_1 will preempt a process in Q_2 .
- Process arrives for Q_0 will, preempt a process in Q_1.

Example of Multilevel Feedback Queue

- **Ex:**
  - Q_0 – time quantum 8 milliseconds
  - Q_1 – time quantum 16 milliseconds
  - Q_2 – FCFS
- **Scheduling**
  - A new job enters Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.